Corrigé détaillé — Lab tcpdump

Cours : 420-B01-HU — Surveillance préventive en cybersécurité
Document : corrigé de l'enseignant, démarche complète
Fichier analysé : `capture_incident.pcap` (440 paquets)
Validé avec : tcpdump 4.99.4 / libpcap 1.10.4

> ⚠ **Confidentiel — enseignant seulement.** Ne pas déposer ce document ni le script `generer_capture.py` dans le dépôt public SnailLabs : ils contiennent toutes les réponses.

Comment utiliser ce corrigé

Pour chaque question, le corrigé donne :

- la **réponse attendue**, dans un encadré ;
- la **démarche**, étape par étape, telle qu'un étudiant devrait la suivre ;
- la **décomposition de chaque commande**, option par option ;
- la **sortie réelle** obtenue sur la capture (copiée telle quelle) ;
- l'**interprétation** et les **pièges fréquents** à surveiller pendant le lab.

Dans toutes les commandes, on suppose que l'étudiant a défini une variable pour abréger le nom du fichier :

```
P=capture_incident.pcap
```

Une variable shell se définit sans espace autour du `=` et s'utilise avec `$P`. Les étudiants peuvent aussi taper le nom complet du fichier à chaque fois.

Partie 0 — Rappels utiles avant de corriger

Lire une ligne de sortie de tcpdump

Presque toutes les réponses de la partie B reposent sur le découpage d'une ligne en colonnes avec `awk`. Il faut donc bien connaître la position de chaque champ. Avec l'option `-n`, une requête DNS s'affiche ainsi :

```
10:00:00.000000 IP 192.168.50.21.61095 > 192.168.50.10.53: 1727+ A? outlook.office365.com. (39)
```

Colonne awk	Contenu	Exemple
`$1`	Heure de capture (microsecondes)	`10:00:00.000000`
`$2`	Protocole de couche 3	`IP`

Colonne awk	Contenu	Exemple
`$3`	Source : adresse IP suivie du port	`192.168.50.21.61095`
`$4`	Flèche de direction	`>`
`$5`	Destination : IP, port, deux-points	`192.168.50.10.53:`
`$6`	DNS : identifiant de transaction (+ = récursion demandée)	`1727+`
`$7`	DNS : type d'enregistrement demandé	`A?`
`$8`	DNS : nom demandé (avec le point final)	`outlook.office365.com.`
dernière	Taille des données DNS, entre parenthèses	`(39)`

Deux détails piègent souvent les étudiants :

- tcpdump colle le **port à l'adresse IP avec un point** (`192.168.50.21.61095`). Pour ne garder que l'adresse, on garde les quatre premiers champs séparés par des points : `cut -d. -f1-4`.
- Les colonnes `$6` et suivantes **changent selon le protocole**. Pour TCP, `$6` est le mot `Flags` et `$7` les drapeaux. Il faut toujours vérifier avec `head` avant de construire un pipeline.

Les options de tcpdump utilisées dans ce corrigé

Option	Rôle	Pourquoi on s'en sert ici
`-r fichier`	Lit un fichier pcap au lieu de capturer	Analyse hors ligne, sans `sudo`
`-n`	N'effectue aucune résolution DNS inverse	Sortie rapide, adresses IP brutes, pas de trafic parasite
`-nn`	Ne traduit pas non plus les ports en noms	Affiche `80` au lieu de `http`
`-c N`	S'arrête après N paquets	Voir le premier paquet seulement
`-tttt`	Horodatage complet avec la date	Bornes de la capture, chronologie
`-ttt`	Délai depuis le paquet affiché précédent	Mesurer l'intervalle des balises C2
`-A`	Affiche la charge utile en ASCII	Lire HTTP, FTP, identifiants
`-X`	Affiche la charge utile en hexadécimal + ASCII	Reconnaître une signature de fichier (PK)
`-e`	Affiche les adresses MAC (couche 2)	Identifier un poste sur le commutateur
`-v`, `-vv`	Plus de détails sur les en-têtes	TTL, sommes de contrôle, question DNS dans la réponse

Option	Rôle	Pourquoi on s'en sert ici
`-w fichier`	Écrit les paquets filtrés dans un nouveau pcap	Extraire une preuve (B6)

Remarque : dans les commandes de ce corrigé, `-nr` est simplement `-n` et `-r` regroupés. L'ordre compte : `-r` doit être suivi immédiatement du nom de fichier, donc `-nr $P` fonctionne mais `-rn $P` échoue (tcpdump chercherait un fichier nommé `n`).

Les outils du shell utilisés

Commande	Rôle dans un pipeline
`wc -l`	Compte les lignes, donc les paquets
`head -N` / `tail -N`	Garde les N premières / dernières lignes
`awk '{print $3}'`	Extrait une colonne (champs séparés par des espaces)
`cut -d. -f1-4`	Découpe selon le point et garde les champs 1 à 4
`sort`	Trie les lignes ; nécessaire avant `uniq`
`uniq -c`	Fusionne les lignes identiques consécutives et les compte
`sort -rn`	Trie numériquement (`-n`), du plus grand au plus petit (`-r`)
`grep motif`	Garde les lignes qui contiennent le motif
`grep -E`	Active les expressions régulières étendues (alternatives avec la barre verticale)
`grep -o`	N'affiche que la partie qui correspond au motif
`grep -c`	Compte les lignes qui correspondent
`tr a-z A-Z`	Convertit les minuscules en majuscules
`tr -d '\n'`	Supprime les retours à la ligne (colle les lignes)
`base32 -d`	Décode une chaîne en base32

Les filtres BPF utilisés

Un filtre BPF (Berkeley Packet Filter) se place **à la fin** de la commande, **entre apostrophes** pour que le shell n'interprète pas les caractères spéciaux (`&`, la barre verticale, `[`, `!`, les parenthèses).

Filtre	Sens
`ip`	Seulement les paquets IPv4 (exclut ARP, IPv6)
`host A`	A est la source ou la destination

Filtre	Sens
`src host A` / `dst host A`	A est la source / la destination
`port N`, `src port N`, `dst port N`	Port TCP ou UDP, dans un sens ou les deux
`tcp`, `udp`, `icmp`	Protocole
`and`, `or`, `not`	Combinaisons logiques
`icmp[icmptype] == icmp-echo`	Requêtes ping (type ICMP 8)
`icmp[icmptype] == icmp-echoreply`	Réponses ping (type ICMP 0)
`tcp[tcpflags] == tcp-syn`	Drapeaux exactement égaux à SYN seul
`tcp[tcpflags] & tcp-push != 0`	Tout paquet dont le drapeau PUSH est levé (paquets de données)

Comprendre `tcp[tcpflags]`

`tcp[tcpflags]` désigne l'octet numéro 13 de l'en-tête TCP, celui qui contient les drapeaux (on peut aussi écrire `tcp[13]`). Chaque drapeau est un bit : FIN = 1, SYN = 2, RST = 4, PSH = 8, ACK = 16.

- `tcp[tcpflags] == tcp-syn` est une **égalité stricte** : l'octet vaut exactement 2. Un SYN-ACK (2 + 16 = 18) ne passe pas. C'est ce qu'on veut pour isoler les tentatives d'ouverture.
- `tcp[tcpflags] & tcp-rst != 0` est un **masque** : on garde le paquet dès que le bit RST est levé, quels que soient les autres. Un RST-ACK passe.
- Pour un SYN-ACK exact, on combine les deux bits avec la barre verticale (OU binaire) : `'tcp[tcpflags] == tcp-syn|tcp-ack'`. Les apostrophes sont obligatoires, sinon le shell prend la barre verticale pour un tube (pipe).

Vérification : `tcpdump -nr $P 'tcp[13] == 2 and src port 61337' | wc -l` donne aussi **18**, comme la version avec `tcp-syn`.

Partie A — Prise en main en direct

Les résultats de la partie A dépendent de la VM de chaque étudiant. Le corrigé indique donc les commandes attendues et ce que l'étudiant doit observer.

A1. Lister les interfaces

```
ip -br a
sudo tcpdump -D
```

- `ip -br a` : `ip address` en format bref (`-br`), une ligne par interface avec son état et son adresse.
- `tcpdump -D` : liste les interfaces que libpcap peut écouter, numérotées. On y voit aussi `any` (toutes les interfaces) et `lo` (boucle locale).

- **Attendu :** l'étudiant repère l'interface qui porte une adresse du cyber range et l'état `UP`, généralement `eth0` sur Kali.

A2. Capturer 10 paquets sans résolution de noms

```
sudo tcpdump -i eth0 -n -c 10
```

- `sudo` : la capture en direct exige les privilèges administrateur (accès brut à la carte réseau).
- `-i eth0` : interface d'écoute.
- `-n` : pas de résolution inverse.
- `-c 10` : arrêt automatique après 10 paquets.
- **Attendu sans `-n` :** les adresses sont remplacées par des noms quand c'est possible, l'affichage est plus lent, et tcpdump **génère lui-même des requêtes DNS** pour résoudre ce qu'il voit. Ces requêtes apparaissent ensuite dans la capture : c'est l'outil qui pollue sa propre mesure. C'est la raison pour laquelle on travaille toujours avec `-n` en analyse.

A3. Observer DNS et HTTP en clair

Terminal 1 :

```
sudo tcpdump -i eth0 -n 'port 53 or port 80'
```

Terminal 2 :

```
curl -s http://example.com > /dev/null
```

- `'port 53 or port 80'` : DNS (port 53) ou HTTP (port 80), dans les deux sens.
- `curl -s` : mode silencieux ; `> /dev/null` jette la page reçue.
- **Attendu :** une requête `A? example.com.` (et souvent `AAAA?` pour IPv6), sa réponse avec l'adresse IP, puis la poignée de main TCP en trois temps (`[S]`, `[S.]`, `[.]`) vers le port 80.

On relance ensuite la capture avec `-A` :

```
sudo tcpdump -i eth0 -n -A 'port 80'
```

- **Attendu :** les lignes `GET / HTTP/1.1`, `Host: example.com` et `User-Agent: curl/x.y.z` sont lisibles en clair dans la charge utile.

A4. Comparer avec HTTPS

```
sudo tcpdump -i eth0 -n -A 'port 53 or port 443'
curl -s https://example.com > /dev/null
```

- **Attendu :** la requête DNS est toujours lisible. Le *Client Hello* TLS laisse voir le nom `example.com` (extension SNI), puis tout le reste est illisible.
- **Point de discussion :** le chiffrement protège le **contenu**, pas les **métadonnées**. On voit encore qui parle à qui, quand, combien et, grâce au DNS et au SNI, vers quel site. La partie B montre que ces métadonnées suffisent pour détecter un C2.

A5. Enregistrer puis relire

```
sudo timeout 30 tcpdump -i eth0 -n -w ma_capture.pcap
tcpdump -n -r ma_capture.pcap | head
```

- `timeout 30` : tue la commande après 30 secondes (sinon il faut un Ctrl+C).
- `-w ma_capture.pcap` : écrit les paquets bruts. **Rien ne s'affiche à l'écran** pendant ce temps, ce qui surprend souvent les étudiants : c'est normal.
- `-r` : relit le fichier ; pas besoin de `sudo`.
- **Piège :** `cat ma_capture.pcap` affiche du charabia, car le format pcap est binaire.

Partie B — L'enquête

B1. Vue d'ensemble

Question 1 — Nombre de paquets

> **Réponse attendue : 440 paquets.**

Démarche

1. Lire le fichier et compter les lignes produites (une ligne par paquet) :

   ```
   tcpdump -nr $P | wc -l
   ```

2. Ignorer le message `reading from file capture_incident.pcap, link-type EN10MB (Ethernet)...`. tcpdump l'écrit sur la sortie d'erreur et non sur la sortie standard, donc `wc -l` ne le compte pas.

Décomposition

- `-n` et `-r $P` : lecture sans résolution de noms.
- `| wc -l` : le tube envoie la sortie à `wc`, qui compte les lignes.

Sortie

```
440
```

Pièges fréquents

- Utiliser l'option `-v` : certains paquets s'affichent alors sur **deux lignes**, et le compte est faux. Pour compter, on n'utilise jamais `-v`.
- Variante acceptable : `capinfos capture_incident.pcap` (outil de Wireshark), s'il est installé.

Question 2 — Début et fin de la capture

> **Réponse attendue : le 13 octobre 2026, de 10:00:00 à 10:04:28.** La capture dure environ 4 minutes 28 secondes.

Démarche

1. Afficher le premier paquet avec la date complète :

```
tcpdump -nr $P -tttt -c 1
```

2. Afficher le dernier paquet :

```
tcpdump -nr $P -tttt | tail -1
```

Décomposition

- `-tttt` : remplace l'heure seule par `AAAA-MM-JJ HH:MM:SS.microsecondes`. Sans cette option, la date n'apparaît pas.
- `-c 1` : s'arrête dès le premier paquet ; plus rapide que `head -1` sur un gros fichier.
- `tail -1` : garde la dernière ligne. Il n'existe pas d'option tcpdump pour « le dernier paquet », d'où le passage par le shell.

Sortie

```
2026-10-13 10:00:00.000000 IP 192.168.50.21.61095 > 192.168.50.10.53: 1727+ A?
outlook.office365.com. (39)
2026-10-13 10:04:28.503764 IP 192.168.50.22.52730 > 52.96.88.130.443: Flags [.], ack 628, win
8192, length 0
```

Pièges fréquents

- L'heure affichée dépend du fuseau horaire de la VM. Si une VM est réglée en UTC, l'étudiant verra 14:00:00 : la réponse est correcte si elle est cohérente. Pour forcer l'heure locale : `TZ=America/Toronto tcpdump -nr $P -tttt -c 1`.

Question 3 — Source la plus bavarde

> **Réponse attendue : 192.168.50.35 (POSTE-ATELIER), avec 136 paquets émis.** C'est plus du double des autres postes (64 et 62), ce qui est anormal pour un poste partagé d'atelier.

Démarche

1. Regarder quelques lignes pour repérer la colonne de la source :

```
tcpdump -nr $P ip | head -3
```

La source est la 3e colonne, avec le port collé à la fin.

2. Extraire la colonne 3, retirer le port, compter les occurrences et trier :

```
tcpdump -nr $P ip | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c | sort -rn
```

Décomposition du pipeline

Étape	Rôle	Exemple de ce qui sort
`tcpdump -nr $P ip`	Lit seulement les paquets IPv4	une ligne complète par paquet
`awk '{print $3}'`	Garde la 3e colonne (source)	`192.168.50.21.61095`

Étape	Rôle	Exemple de ce qui sort
`cut -d. -f1-4`	Coupe aux points, garde les 4 premiers champs	`192.168.50.21`
`sort`	Regroupe les adresses identiques	lignes triées
`uniq -c`	Compte chaque adresse	`62 192.168.50.21`
`sort -rn`	Classe du plus grand au plus petit	le plus bavard en premier

Sortie

```
    136 192.168.50.35
     64 192.168.50.22
     62 192.168.50.21
     36 192.168.50.20
     32 198.51.100.66
     27 192.168.50.10
     24 52.96.88.130
     18 142.250.80.4
     12 52.113.194.132
     12 205.236.11.20
     12 142.251.41.35
      4 93.184.215.201
      1 192.168.50.1
```

Interprétation

- Le poste de l'atelier émet à lui seul près du tiers des paquets.
- On remarque déjà une adresse externe, **198.51.100.66**, qui ne correspond à aucun service connu (Microsoft, Desjardins, Revenu Québec, Google). On y reviendra en B2 et B5.
- 192.168.50.1 (la passerelle) n'émet qu'un paquet : une réponse ping (voir B3).

Pièges fréquents

- Oublier `sort` avant `uniq -c` : `uniq` ne fusionne que les lignes **consécutives**, et les comptes sont alors fragmentés.
- Oublier `cut` : chaque couple IP + port est compté à part, et le classement ne veut rien dire.
- Le filtre `ip` est utile par principe : sur une capture qui contient de l'ARP, la colonne 3 n'est pas une adresse IP.

B2. Le DNS raconte l'histoire

Question 4 — Domaines demandés

> **Réponse attendue :** 20 requêtes DNS. Domaines légitimes : outlook.office365.com (4), www.desjardins.com (3), www.revenuquebec.ca (2), www.google.ca (2), teams.microsoft.com (2), intranet.ebenisterie-gatineau.local (1), ctldl.windowsupdate.com (1). Domaine suspect : **cdn-maj-update.net** et 4 sous-domaines numérotés `0-` à `3-`.

Démarche

1. Garder seulement les **requêtes** (vers le port 53), pas les réponses :

   ```
   tcpdump -nr $P 'udp dst port 53' | head -2
   ```

   Le nom demandé est en 8e colonne.

2. Extraire, compter et trier :

   ```
   tcpdump -nr $P 'udp dst port 53' | awk '{print $8}' | sort | uniq -c | sort -rn
   ```

Décomposition

- `'udp dst port 53'` : UDP dont la **destination** est le port 53, donc les questions envoyées au serveur DNS. Avec `port 53` tout court, on aurait aussi les réponses, dont la colonne 8 contient autre chose (l'adresse IP retournée).
- `awk '{print $8}'` : la colonne du nom demandé (voir le tableau de la partie 0).

Sortie

```
      4 outlook.office365.com.
      3 www.desjardins.com.
      2 www.revenuquebec.ca.
      2 www.google.ca.
      2 teams.microsoft.com.
      1 intranet.ebenisterie-gatineau.local.
      1 ctldl.windowsupdate.com.
      1 cdn-maj-update.net.
      1 3-jnkqzx2.cdn-maj-update.net.
      1 2-s3bnobqwsz.cdn-maj-update.net.
      1 1-xhgllfpbtg.cdn-maj-update.net.
      1 0-izgecr33mr.cdn-maj-update.net.
```

Interprétation

- Le point final (`outlook.office365.com.`) est normal : c'est la racine DNS. tcpdump affiche le nom complet (FQDN).
- Les sous-domaines aléatoires et numérotés sont une signature classique d'**exfiltration ou de tunnel DNS** : les données volées sont cachées dans les noms demandés. On les décode en B5.

Question 5 — Le domaine suspect

> **Réponse attendue : cdn-maj-update.net**, demandé par **192.168.50.35**, résolu vers **198.51.100.66**. Le nom imite un réseau de diffusion (CDN) de mises à jour pour paraître légitime.

Démarche

1. Trouver le poste qui demande ce domaine :

   ```
   tcpdump -nr $P 'udp dst port 53' | grep cdn-maj
   ```

   Toutes les lignes ont `192.168.50.35` comme source.

2. Trouver l'adresse IP retournée. **Piège important :** dans la sortie par défaut, tcpdump n'affiche **pas le nom** dans la réponse, seulement l'adresse. `grep cdn-maj` sur les réponses ne donne donc rien. Il

faut afficher requêtes et réponses du poste ensemble et les associer par leur **identifiant de transaction** :

```
tcpdump -nr $P 'udp port 53 and host 192.168.50.35'
```

La requête `26680+ A? cdn-maj-update.net.` est suivie de la réponse `26680 1/0/0 A 198.51.100.66`. Le même numéro 26680 relie les deux.

3. Méthode plus directe : demander à tcpdump d'afficher la question dans la réponse avec `-vv` :

```
tcpdump -vv -nr $P 'udp src port 53 and dst host 192.168.50.35' | tail -2
```

Décomposition

- `'udp port 53 and host 192.168.50.35'` : tout le DNS, dans les deux sens, impliquant le poste suspect.
- `26680+` : identifiant de transaction ; le `+` signifie « récursion demandée ».
- `1/0/0` : 1 réponse, 0 autorité, 0 enregistrement additionnel.
- `-vv` : mode très détaillé ; affiche `q: A? nom.` (la question) dans la réponse.

Sortie (méthode `-vv`)

```
10:00:51.092089 IP (tos 0x0, ttl 64, id 1, offset 0, flags [none], proto UDP (17), length 98)
    192.168.50.10.53 > 192.168.50.35.63024: [udp sum ok] 26680 q: A? cdn-maj-update.net. 1/0/0
cdn-maj-update.net. A 198.51.100.66 (70)
```

Interprétation

- Le poste utilise le DNS interne (SRV-DC01) pour résoudre le domaine malveillant. C'est une bonne nouvelle pour la défense : les journaux du DNS interne contiennent la même trace, et un blocage au DNS interne est possible.
- 198.51.100.0/24 est une plage réservée à la documentation (RFC 5737). On peut le signaler aux étudiants curieux : elle n'est jamais routée sur Internet, d'où son usage dans un scénario fictif.

B3. Reconnaissance

Question 6 — Le balayage ICMP

> **Réponse attendue : 192.168.50.35** envoie **30 requêtes** ping (192.168.50.1 à .30) entre 10:00:22,57 et 10:00:23,15. **5 hôtes répondent : .1, .10, .20, .21 et .22.**

Démarche

1. Afficher les requêtes ping :

```
tcpdump -nr $P 'icmp[icmptype] == icmp-echo' | head -3
```

On voit les adresses de destination augmenter d'une unité à chaque paquet, toutes les 20 ms.

2. Les compter :

```
tcpdump -nr $P 'icmp[icmptype] == icmp-echo' | wc -l
```

3. Afficher les réponses :

```
tcpdump -nr $P 'icmp[icmptype] == icmp-echoreply'
```

Décomposition

- `icmp[icmptype]` : le premier octet de l'en-tête ICMP, qui contient le type.
- `icmp-echo` (type 8) : requête ping ; `icmp-echoreply` (type 0) : réponse.
- On peut aussi écrire `'icmp[0] == 8'` et `'icmp[0] == 0'`.

Sorties

```
10:00:22.566090 IP 192.168.50.35 > 192.168.50.1: ICMP echo request, id 19229, seq 1, length 8
10:00:22.586090 IP 192.168.50.35 > 192.168.50.2: ICMP echo request, id 19229, seq 2, length 8
10:00:22.606090 IP 192.168.50.35 > 192.168.50.3: ICMP echo request, id 19229, seq 3, length 8
```

```
30

10:00:22.566890 IP 192.168.50.1 > 192.168.50.35: ICMP echo reply, id 19229, seq 1, length 8
10:00:22.746889 IP 192.168.50.10 > 192.168.50.35: ICMP echo reply, id 19229, seq 10, length 8
10:00:22.946889 IP 192.168.50.20 > 192.168.50.35: ICMP echo reply, id 19229, seq 20, length 8
10:00:22.966889 IP 192.168.50.21 > 192.168.50.35: ICMP echo reply, id 19229, seq 21, length 8
10:00:22.986889 IP 192.168.50.22 > 192.168.50.35: ICMP echo reply, id 19229, seq 22, length 8
```

Interprétation

- Un même identifiant ICMP (`id 19229`) pour toutes les requêtes, avec un numéro de séquence égal au dernier octet de l'adresse visée : c'est un outil automatisé (style `nmap -sn` ou un script), pas un humain qui tape `ping`.
- L'attaquant dresse la carte du réseau : il sait maintenant que 5 hôtes sont actifs.

Pièges fréquents

- Le filtre `icmp` seul mélange requêtes et réponses : 35 paquets au lieu de 30.

Question 7 — Le balayage de ports

> **Réponse attendue :**
>
> **Cible :** 192.168.50.20 (SRV-FICHIERS).
>
> **Type :** balayage SYN (*half-open*, comme `nmap -sS`).
>
> **Ports testés :** 18.
>
> **Ports ouverts : 21 (FTP), 22 (SSH), 139 et 445 (SMB), 3389 (RDP).**
>
> **RST :** l'attaquant n'achève jamais la poignée de main, pour aller plus vite et éviter d'être journalisé par l'application.

Démarche

1. Repérer qui envoie beaucoup de SYN, et vers qui :

```
tcpdump -nr $P 'tcp[tcpflags] == tcp-syn' | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c
| sort -rn
```

Résultat : 29 SYN de .35, contre 7 pour chacun des autres postes.

2. Regarder ces SYN de plus près :

```
tcpdump -nr $P 'tcp[tcpflags] == tcp-syn and src host 192.168.50.35' | head -4
```

   À partir de 10:00:27, le port **source** reste fixe (61337), la cible est toujours 192.168.50.20, le port de destination change à chaque paquet et la fenêtre TCP vaut **1024**.

3. Compter les ports testés, en se limitant au port source du balayage :

```
tcpdump -nr $P 'src port 61337 and tcp[tcpflags] == tcp-syn' | wc -l
```

4. Trouver les ports ouverts : ce sont ceux qui répondent par un **SYN-ACK** :

```
tcpdump -nr $P 'dst port 61337 and tcp[tcpflags] == tcp-syn|tcp-ack'
```

5. Comparer un port ouvert et un port fermé :

```
tcpdump -nr $P 'host 192.168.50.20 and port 445 and port 61337'
tcpdump -nr $P 'host 192.168.50.20 and port 3306 and port 61337'
```

Décomposition

- `src port 61337` : le port source choisi par l'outil de balayage. Le garder dans le filtre élimine la vraie session FTP de la question 9.
- `'tcp[tcpflags] == tcp-syn|tcp-ack'` : drapeaux SYN et ACK exactement, c'est-à-dire la réponse « port ouvert ».
- `Flags [S.]` dans la sortie : `S` = SYN, `.` = ACK.
- `Flags [R.]` : RST-ACK, la réponse « port fermé ».

Sorties

Les 5 ports ouverts (étape 4) :

```
10:00:27.172589 IP 192.168.50.20.21 > 192.168.50.35.61337: Flags [S.], seq 859811621, ack
469373947, win 8192, length 0
10:00:27.175589 IP 192.168.50.20.445 > 192.168.50.35.61337: Flags [S.], seq 899742115, ack
1729840774, win 8192, length 0
10:00:27.193589 IP 192.168.50.20.22 > 192.168.50.35.61337: Flags [S.], seq 861286506, ack
1709593514, win 8192, length 0
10:00:27.196589 IP 192.168.50.20.3389 > 192.168.50.35.61337: Flags [S.], seq 1113128903, ack
233484458, win 8192, length 0
10:00:27.199589 IP 192.168.50.20.139 > 192.168.50.35.61337: Flags [S.], seq 927503101, ack
1535342885, win 8192, length 0
```

Port ouvert (445) : SYN, puis SYN-ACK, puis RST de l'attaquant :

```
10:00:27.175089 IP 192.168.50.35.61337 > 192.168.50.20.445: Flags [S], seq 1729840773, win 1024,
length 0
10:00:27.175589 IP 192.168.50.20.445 > 192.168.50.35.61337: Flags [S.], seq 899742115, ack
1729840774, win 8192, length 0
10:00:27.175989 IP 192.168.50.35.61337 > 192.168.50.20.445: Flags [R], seq 1729840774, win 8192,
length 0
```

Port fermé (3306) : SYN, puis RST-ACK du serveur :

```
10:00:27.166089 IP 192.168.50.35.61337 > 192.168.50.20.3306: Flags [S], seq 1971005519, win 1024,
length 0
10:00:27.166589 IP 192.168.50.20.3306 > 192.168.50.35.61337: Flags [R.], seq 0, ack 1971005520,
win 8192, length 0
```

Les 13 ports fermés sont : 23, 25, 53, 80, 110, 135, 143, 443, 993, 1433, 3306, 5900 et 8080. On les obtient avec :

```
tcpdump -nr $P 'src host 192.168.50.20 and dst port 61337 and tcp[tcpflags] & tcp-rst != 0'
```

Interprétation

- **Balayage SYN :** l'outil envoie un SYN, lit la réponse, puis coupe avec un RST sans envoyer l'ACK final. La connexion n'est jamais complète, donc les applications (FTP, SSH) ne la journalisent généralement pas. Seul un pare-feu, un IDS ou une capture réseau la voient.
- **Signatures de nmap :** port source fixe, fenêtre TCP de 1024, 18 SYN en environ 50 ms, ports testés dans un ordre aléatoire.
- **Ce que l'attaquant apprend :** le serveur expose FTP (en clair), SSH, le partage de fichiers Windows et le bureau à distance. La question 9 montre qu'il exploite immédiatement le FTP.

Pièges fréquents

- Filtrer `'src host 192.168.50.20 and tcp[tcpflags] == tcp-syn|tcp-ack'` sans le port 61337 : on obtient **7 lignes** au lieu de 5. Les deux de trop (ports 21 et 50001 vers les ports 52100 et 52101) appartiennent à la vraie session FTP de la question 9. Un étudiant qui annonce « 50001 ouvert » n'a pas distingué le balayage de la connexion légitime : c'est une bonne occasion de discuter.
- Oublier les apostrophes autour du filtre : le shell coupe la commande à la barre verticale et affiche une erreur.

B4. Identifiants en clair

Question 8 — Connexion à l'intranet

> **Réponse attendue :** utilisateur **mlavoie**, mot de passe **Erable2026!** (`%21` est le point d'exclamation encodé en URL). La connexion part de **192.168.50.22 (POSTE-RH)** à 10:00:17.

Démarche

1. Lister les échanges HTTP avec l'intranet (SRV-DC01, port 80) :

   ```
   tcpdump -nr $P 'tcp port 80 and host 192.168.50.10' | grep HTTP
   ```

   On voit un `GET /connexion.php`, la page de connexion (`200 OK`), puis un `POST /connexion.php` : c'est l'envoi du formulaire.

2. Afficher le contenu en ASCII et chercher le formulaire :

   ```
   tcpdump -nr $P -A 'tcp port 80 and host 192.168.50.10' | grep -E 'POST|utilisateur=|Set-
   Cookie'
   ```

3. Décoder `%21`. Les étudiants peuvent utiliser une table ASCII (0x21 = `!`) ou :

```
python3 -c "import urllib.parse; print(urllib.parse.unquote('Erable2026%21'))"
```

Décomposition

- `-A` : affiche la charge utile en ASCII. Sans `-A`, on ne voit que les en-têtes TCP.
- `grep -E 'POST|utilisateur=|Set-Cookie'` : trois motifs en alternative, grâce aux expressions étendues.
- Dans un formulaire HTML envoyé par POST, les champs sont codés comme `nom=valeur&nom2=valeur2` dans le corps de la requête.

Sorties

```
10:00:17.466090 IP 192.168.50.22.51022 > 192.168.50.10.80: Flags [P.], seq 1:130, ack 1, win 8192,
length 129: HTTP: GET /connexion.php HTTP/1.1
10:00:17.486090 IP 192.168.50.10.80 > 192.168.50.22.51022: Flags [P.], seq 1:179, ack 130, win
8192, length 178: HTTP: HTTP/1.1 200 OK
10:00:17.506090 IP 192.168.50.22.51022 > 192.168.50.10.80: Flags [P.], seq 130:336, ack 179, win
8192, length 206: HTTP: POST /connexion.php HTTP/1.1
10:00:17.526090 IP 192.168.50.10.80 > 192.168.50.22.51022: Flags [P.], seq 179:269, ack 336, win
8192, length 90: HTTP: HTTP/1.1 302 Found
```

```
...POST /connexion.php HTTP/1.1
utilisateur=mlavoie&motdepasse=Erable2026%21&valider=Connexion
Set-Cookie: PHPSESSID=7f3a9c21be; HttpOnly
```

Interprétation

- La réponse `302 Found` (redirection vers `/accueil.php`) confirme que la connexion a **réussi** : les identifiants sont valides.
- Le serveur remet un cookie de session sans l'attribut `Secure`. Quiconque capture ce cookie peut usurper la session sans connaître le mot de passe (détournement de session).
- **Lien avec l'incident :** cette connexion n'est pas une attaque. C'est une faiblesse de configuration. Mais le poste .35 étant compromis et sur le même réseau, l'attaquant pourrait capturer ces identifiants de la même façon.

Pièges fréquents

- Rendre `Erable2026%21` sans le décoder : réponse partielle.

Question 9 — La session FTP

> **Réponse attendue :**
>
> **Compte :** `sauvegarde` / **mot de passe :** `Bkp!Atelier99` (compte de service, utilisé depuis un poste de travail).
>
> **Fichier :** `paie_2026_T3.xlsx` : données de paie du 3e trimestre, donc renseignements personnels (NAS, salaires, coordonnées bancaires).
>
> **Canal de données :** port **50001** = 195 × 256 + 81.

Démarche

1. N'afficher que les paquets de la session FTP qui transportent des données (drapeau PUSH). tcpdump décode le protocole FTP et affiche la commande en fin de ligne :

```
tcpdump -nr $P 'tcp port 21 and tcp[tcpflags] & tcp-push != 0'
```

2. Variante avec `-A`, plus proche de ce que les étudiants ont pratiqué :

```
tcpdump -nr $P -A 'tcp port 21' | grep -E 'USER|PASS|RETR'
```

3. Lire la réponse `227 Entering Passive Mode (192,168,50,20,195,81)` et calculer le port.
4. Vérifier que le transfert a bien eu lieu sur ce port :

```
tcpdump -nr $P 'port 50001'
tcpdump -nr $P -X 'port 50001 and tcp[tcpflags] & tcp-push != 0' | head -4
```

Décomposition

- `tcp[tcpflags] & tcp-push != 0` : garde les paquets qui portent des données applicatives et élimine les accusés de réception vides. La sortie devient beaucoup plus lisible.
- `-X` : hexadécimal + ASCII, utile pour reconnaître une signature de fichier.
- Le mode passif FTP : le client envoie `PASV`, le serveur répond par 6 nombres. Les 4 premiers sont l'adresse IP, les 2 derniers le port, à calculer ainsi : **port = 1er × 256 + 2e**.

Sortie (étape 1)

```
10:00:33.250089 IP 192.168.50.20.21 > 192.168.50.35.52100: Flags [P.], ... length 27: FTP: 220
SRV-FICHIERS FTP pret
10:00:33.270089 IP 192.168.50.35.52100 > 192.168.50.20.21: Flags [P.], ... length 17: FTP: USER
sauvegarde
10:00:33.290089 IP 192.168.50.20.21 > 192.168.50.35.52100: Flags [P.], ... length 41: FTP: 331 Mot
de passe requis pour sauvegarde
10:00:33.310089 IP 192.168.50.35.52100 > 192.168.50.20.21: Flags [P.], ... length 20: FTP: PASS
Bkp!Atelier99
10:00:33.330089 IP 192.168.50.20.21 > 192.168.50.35.52100: Flags [P.], ... length 26: FTP: 230
Utilisateur connecte
10:00:33.350089 IP 192.168.50.35.52100 > 192.168.50.20.21: Flags [P.], ... length 6: FTP: PASV
10:00:33.370089 IP 192.168.50.20.21 > 192.168.50.35.52100: Flags [P.], ... length 50: FTP: 227
Entering Passive Mode (192,168,50,20,195,81)
10:00:33.390089 IP 192.168.50.35.52100 > 192.168.50.20.21: Flags [P.], ... length 24: FTP: RETR
paie_2026_T3.xlsx
10:00:33.410089 IP 192.168.50.20.21 > 192.168.50.35.52100: Flags [P.], ... length 42: FTP: 150
Ouverture de la connexion de donnees
10:00:33.430089 IP 192.168.50.20.21 > 192.168.50.35.52100: Flags [P.], ... length 23: FTP: 226
Transfert termine
10:00:33.450089 IP 192.168.50.35.52100 > 192.168.50.20.21: Flags [P.], ... length 6: FTP: QUIT
10:00:33.470089 IP 192.168.50.20.21 > 192.168.50.35.52100: Flags [P.], ... length 15: FTP: 221 Au
revoir
```

(Les numéros de séquence ont été remplacés par `...` pour la lisibilité.)

Calcul du port de données

```
(192,168,50,20,195,81)
adresse = 192.168.50.20
port    = 195 × 256 + 81 = 49 920 + 81 = 50 001
```

Sortie (étape 4, extrait `-X`)

```
10:00:33.240089 IP 192.168.50.20.50001 > 192.168.50.35.52101: Flags [P.], seq 232243674:232244578,
ack 59514464, win 8192, length 904
     0x0000:  4500 03b0 0001 0000 4006 91bf c0a8 3214  E.......@.....2.
     0x0010:  c0a8 3223 c351 cb85 0dd7 c1da 038c 1e60  ..2#.Q.........`
     0x0020:  5018 2000 8a1c 0000 504b 0304 eae6 649b  P.......PK....d.
```

Interprétation

- Les octets `50 4b 03 04` (`PK..`) sont la signature d'une archive ZIP. Un fichier `.xlsx` est une archive ZIP : la signature confirme qu'un vrai fichier Excel a été transféré (904 octets).
- Codes FTP utiles : `220` serveur prêt, `331` mot de passe requis, `230` connexion réussie, `227` mode passif, `150` ouverture du transfert, `226` transfert terminé, `221` fin de session.
- **Gravité :** vol de renseignements personnels. Au Québec, la Loi 25 oblige l'entreprise à évaluer le risque de préjudice et, s'il est sérieux, à aviser la Commission d'accès à l'information et les personnes concernées.
- Un compte de service (`sauvegarde`) utilisé depuis un poste de travail est en soi une anomalie à signaler.

Pièges fréquents

- Additionner au lieu de multiplier (195 + 81 = 276) ou inverser les deux nombres (81 × 256 + 195).
- Avec `-A` seul, chaque ligne de charge utile est précédée de quelques caractères illisibles (le reste de l'en-tête affiché en ASCII). C'est normal ; la commande FTP est à la fin de la ligne.

B5. Exfiltration et commande-contrôle

Question 10 — L'exfiltration DNS

> **Réponse attendue :**
>
> **Chaîne reconstituée :** `izgecr33mrxhgllfpbtgs3bnobqwszjnkqzx2`
>
> **Drapeau : FLAG{dns-exfil-paie-T3}**

Démarche

1. Afficher les requêtes DNS du poste suspect vers le domaine malveillant :

   ```
   tcpdump -nr $P 'udp dst port 53 and src host 192.168.50.35' | grep cdn-maj
   ```

   Quatre requêtes à 1,7 s d'intervalle (10:00:41 à 10:00:46), numérotées de 0 à 3.

2. Extraire uniquement les sous-domaines numérotés et les trier :

   ```
   tcpdump -nr $P 'udp dst port 53' | grep -o '[0-9]-[a-z0-9]*\.cdn-maj' | sort
   ```

3. Retirer le numéro et le domaine pour ne garder que les données :

   ```
   tcpdump -nr $P 'udp dst port 53' | grep -o '[0-9]-[a-z0-9]*\.cdn-maj' | sort | cut -d- -f2 |
   cut -d. -f1
   ```

4. Coller les morceaux, passer en majuscules, ajouter le bourrage et décoder :

```
S=$(tcpdump -nr $P 'udp dst port 53' | grep -o '[0-9]-[a-z0-9]*\.cdn-maj' \
    | sort | cut -d- -f2 | cut -d. -f1 | tr -d '\n' | tr a-z A-Z)
echo "$S"
echo ${#S}
echo "$S===" | base32 -d
```

Décomposition

Élément	Rôle
`grep -o '[0-9]-[a-z0-9]*\.cdn-maj'`	Extrait seulement « chiffre, tiret, lettres et chiffres, .cdn-maj »
`\.` dans le motif	Un vrai point (sans la barre oblique, le point veut dire « n'importe quel caractère »)
`sort`	Remet les morceaux dans l'ordre 0, 1, 2, 3 grâce au numéro en tête
`cut -d- -f2`	Coupe au tiret et garde ce qui suit le numéro
`cut -d. -f1`	Coupe au point et garde le morceau de données
`tr -d '\n'`	Colle les 4 lignes en une seule chaîne
`tr a-z A-Z`	Passe en majuscules : l'alphabet base32 standard est A à Z et 2 à 7
`S=$( ... )`	Range le résultat dans la variable S
`${#S}`	Longueur de la chaîne (37 caractères)
`"$S==="`	Ajoute 3 caractères de bourrage
`base32 -d`	Décode

Sorties

```
0-izgecr33mr.cdn-maj
1-xhgllfpbtg.cdn-maj
2-s3bnobqwsz.cdn-maj
3-jnkqzx2.cdn-maj
```

```
IZGECR33MRXHGLLFPBTGS3BNOBQWSZJNKQZX2
37
FLAG{dns-exfil-paie-T3}
```

Pourquoi trois `=` ?

Le base32 code les données par blocs de **8 caractères**. Une chaîne valide a donc une longueur multiple de 8. Ici, 37 caractères : le multiple de 8 suivant est 40, il manque **3 caractères** de bourrage. L'attaquant les a retirés parce que le signe `=` est interdit dans un nom de domaine.

Sans le bourrage, `base32 -d` décode le début puis s'arrête sur une erreur :

```
FLAG{dns-exfil-paie-base32: invalid input
```

Un étudiant qui rend `FLAG{dns-exfil-paie-` a fait presque tout le travail : il lui manque la notion de bourrage.

Interprétation

- Le DNS est rarement bloqué en sortie, et ses requêtes passent même par le serveur DNS interne, qui les relaie vers Internet. C'est un canal d'exfiltration discret.
- Les signes d'alerte : sous-domaines longs et aléatoires, numérotés, nombreux vers un même domaine récent, à intervalle régulier.
- La chaîne en minuscules sans bourrage est typique : les noms de domaine ne distinguent pas majuscules et minuscules.

Pièges fréquents

- Ne pas trier : si les morceaux sont collés dans le désordre, le décodage donne du charabia.
- Oublier `tr a-z A-Z` : selon la version de `base32`, les minuscules sont refusées.
- Écrire `grep -o '[0-9]-.*cdn-maj'` : le `.*` est gourmand et peut avaler plus que prévu. Le motif `[a-z0-9]*` est plus sûr.

Question 11 — Le canal de commande-contrôle (C2)

> **Réponse attendue :**
>
> **Serveur :** 198.51.100.66, port TCP **8080**.
>
> **Intervalle :** environ **30 secondes** (entre 29,65 et 30,38 s) : c'est du **beaconing** (balise).
>
> **User-Agent :** `Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)`, soit Internet Explorer 6 sous Windows XP, impossible sur un poste de 2026.
>
> **Réponse différente :** la 6e balise (`n=5`) reçoit **`CMD:whoami`** au lieu de `NOOP`.

Démarche

1. Voir avec qui 198.51.100.66 communique :

   ```
   tcpdump -nr $P 'host 198.51.100.66' | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c
   ```

   Seul 192.168.50.35 lui parle.

2. Afficher chaque ouverture de connexion vers ce serveur, avec la date :

   ```
   tcpdump -nr $P -tttt 'host 198.51.100.66 and tcp[tcpflags] == tcp-syn'
   ```

3. Mesurer directement l'intervalle entre deux connexions avec `-ttt` :

   ```
   tcpdump -nr $P -ttt 'host 198.51.100.66 and tcp[tcpflags] == tcp-syn'
   ```

4. Lire le contenu des échanges HTTP :

   ```
   tcpdump -nr $P -A 'tcp port 8080' | grep -E 'GET|User-Agent|NOOP|CMD:'
   ```

5. Isoler la réponse différente :

   ```
   tcpdump -nr $P -A 'src host 198.51.100.66 and tcp[tcpflags] & tcp-push != 0' | grep -E
   '^(NOOP|CMD)' | sort | uniq -c
   ```

Décomposition

- `-ttt` : au lieu de l'heure, affiche le **délai depuis le paquet affiché précédent**. Combiné au filtre sur les SYN, il donne directement l'intervalle entre deux balises.
- `grep -E '^(NOOP|CMD)'` : le `^` signifie « en début de ligne » ; la parenthèse regroupe les deux alternatives.

Sorties

```
2026-10-13 10:00:51.130089 IP 192.168.50.35.53000 > 198.51.100.66.8080: Flags [S], seq 1516953726,
win 64240, length 0
2026-10-13 10:01:21.280033 IP 192.168.50.35.53001 > 198.51.100.66.8080: Flags [S], seq 1345484360,
win 64240, length 0
2026-10-13 10:01:51.661807 IP 192.168.50.35.53002 > 198.51.100.66.8080: Flags [S], seq 754776613,
win 64240, length 0
2026-10-13 10:02:21.372478 IP 192.168.50.35.53003 > 198.51.100.66.8080: Flags [S], seq 255057538,
win 64240, length 0
2026-10-13 10:02:51.125553 IP 192.168.50.35.53004 > 198.51.100.66.8080: Flags [S], seq 1660196971,
win 64240, length 0
2026-10-13 10:03:20.845963 IP 192.168.50.35.53005 > 198.51.100.66.8080: Flags [S], seq 468396278,
win 64240, length 0
2026-10-13 10:03:50.499141 IP 192.168.50.35.53006 > 198.51.100.66.8080: Flags [S], seq 1308762066,
win 64240, length 0
2026-10-13 10:04:20.233764 IP 192.168.50.35.53007 > 198.51.100.66.8080: Flags [S], seq 677912749,
win 64240, length 0
```

```
 00:00:00.000000 IP 192.168.50.35.53000 > 198.51.100.66.8080: Flags [S], ...
 00:00:30.149944 IP 192.168.50.35.53001 > 198.51.100.66.8080: Flags [S], ...
 00:00:30.381774 IP 192.168.50.35.53002 > 198.51.100.66.8080: Flags [S], ...
 00:00:29.710671 IP 192.168.50.35.53003 > 198.51.100.66.8080: Flags [S], ...
 00:00:29.753075 IP 192.168.50.35.53004 > 198.51.100.66.8080: Flags [S], ...
 00:00:29.720410 IP 192.168.50.35.53005 > 198.51.100.66.8080: Flags [S], ...
 00:00:29.653178 IP 192.168.50.35.53006 > 198.51.100.66.8080: Flags [S], ...
 00:00:29.734623 IP 192.168.50.35.53007 > 198.51.100.66.8080: Flags [S], ...
```

```
GET /maj/check?id=ATELIER-35&n=0 HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
NOOP
...
GET /maj/check?id=ATELIER-35&n=5 HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
CMD:whoami
...
```

```
      1 CMD:whoami
      7 NOOP
```

Interprétation

- **Beaconing :** l'implant appelle son serveur à intervalle régulier (ici 30 s avec une petite variation aléatoire, le *jitter*, pour échapper aux détections trop simples) pour demander s'il y a du travail. `NOOP` signifie « rien à faire ».
- **`CMD:whoami` :** l'opérateur passe à l'action. `whoami` affiche le compte sous lequel tourne l'implant : c'est typiquement la première commande d'un attaquant pour savoir quels privilèges il a obtenus.

- **Autres indices :** le paramètre `id=ATELIER-35` identifie la victime auprès du serveur ; le port 8080 en HTTP clair est inhabituel pour une sortie Internet ; le nom `/maj/check` imite une vérification de mise à jour.
- **Lien avec le DNS :** la requête `cdn-maj-update.net` de 10:00:51,08 précède de 50 ms la première balise (10:00:51,13). L'implant résout le domaine puis se connecte.

Pièges fréquents

- Avec `-ttt` sans filtre sur les SYN, chaque paquet de la session est affiché et l'écart mesuré est de quelques millisecondes. Le filtre doit garder **un paquet par connexion**.

B6. Extraire la preuve

Question 12 — Créer `preuve_c2.pcap`

> **Réponse attendue :** `preuve_c2.pcap` contient **80 paquets** (8 connexions de 10 paquets).

Démarche

1. Écrire les paquets filtrés dans un nouveau fichier :

```
tcpdump -nr $P -w preuve_c2.pcap 'host 198.51.100.66'
```

2. Vérifier le contenu du nouveau fichier :

```
tcpdump -nr preuve_c2.pcap | wc -l
```

Décomposition

- `-r $P` et `-w preuve_c2.pcap` ensemble : tcpdump lit un fichier, applique le filtre et écrit le résultat dans un autre. Le fichier d'origine n'est pas modifié.
- `'host 198.51.100.66'` : tout ce qui va vers le serveur C2 ou en vient.

Sortie

```
80
```

Détail : chaque connexion compte 3 paquets pour la poignée de main, 2 pour la requête et son accusé, 2 pour la réponse et son accusé, 3 pour la fermeture, soit 10 paquets.

Interprétation

- En réponse aux incidents, on ne transmet pas une capture complète quand un extrait suffit : on limite la diffusion des données sans lien avec l'incident (ici, les identifiants de POSTE-RH, par exemple).
- Bonne pratique à mentionner : calculer une empreinte du fichier extrait pour la chaîne de possession :

```
sha256sum preuve_c2.pcap
```

Pièges fréquents

- Placer `-w` après le filtre : `tcpdump -nr $P 'host 198.51.100.66' -w preuve_c2.pcap` fonctionne aussi sous Linux (testé avec la version 4.99), car la bibliothèque GNU réordonne les options. Ce n'est pas garanti sur d'autres systèmes : la convention reste **les options d'abord, le filtre à la fin**.
- Ajouter `-A` ou `-v` en même temps que `-w` : ces options ne changent rien au fichier écrit, qui contient toujours les paquets bruts.
- Écraser la capture d'origine en se trompant de nom de fichier après `-w`.

Partie C — Rapport : éléments attendus

Chronologie de l'incident

Heure	Poste	Événement
10:00:00	Tous	Trafic normal : Microsoft 365, Desjardins, Revenu Québec, Teams, Google
10:00:14	.21 POSTE-COMPTA	Mise à jour Windows en HTTP (trafic légitime)
10:00:17	.22 POSTE-RH	Connexion à l'intranet en HTTP clair ; identifiants de mlavoie exposés
10:00:22	.35 POSTE-ATELIER	Balayage ICMP de 192.168.50.1 à .30 ; 5 hôtes actifs
10:00:27	.35 POSTE-ATELIER	Balayage SYN de SRV-FICHIERS ; 5 ports ouverts sur 18
10:00:33	.35 POSTE-ATELIER	FTP avec le compte sauvegarde ; vol de paie_2026_T3.xlsx
10:00:41	.35 POSTE-ATELIER	Exfiltration DNS vers cdn-maj-update.net (4 requêtes)
10:00:51	.35 POSTE-ATELIER	Première balise C2 vers 198.51.100.66:8080
10:03:20	.35 POSTE-ATELIER	Le C2 envoie la commande whoami
10:04:20	.35 POSTE-ATELIER	8e balise ; fin de la capture à 10:04:28

Indicateurs de compromission (IoC)

Type	Valeur
Adresse IP	198.51.100.66
Domaine	cdn-maj-update.net (et ses sous-domaines)
Port / protocole	TCP 8080, HTTP en clair
URL	/maj/check?id=ATELIER-35
User-Agent	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
Comportement	Balise toutes les 30 secondes environ
Poste compromis	192.168.50.35 (POSTE-ATELIER)
Comptes exposés	sauvegarde (FTP), mlavoie (intranet)
Donnée volée	paie_2026_T3.xlsx

Recommandations

Accepter toute recommandation justifiée et priorisée. Exemples attendus :

1. **Immédiat (endiguement) :**
   - isoler POSTE-ATELIER du réseau, sans l'éteindre (préserver la mémoire pour l'analyse) ;
   - bloquer 198.51.100.66 et cdn-maj-update.net au pare-feu et au DNS interne ;
   - réinitialiser les mots de passe de `sauvegarde` et `mlavoie`, et invalider les sessions de l'intranet ;
   - conserver la capture et son empreinte SHA-256 comme preuve.
2. **Court terme :**
   - évaluer l'incident de confidentialité (Loi 25) : risque de préjudice, registre, avis à la CAI et aux employés si le risque est sérieux ;
   - remplacer FTP par SFTP et forcer HTTPS sur l'intranet, avec l'attribut `Secure` sur les cookies ;
   - interdire l'usage du compte `sauvegarde` depuis les postes de travail.
3. **Moyen terme :**
   - segmenter le réseau : le poste de l'atelier n'a pas à joindre le serveur de fichiers en FTP ni à balayer le réseau ;
   - n'autoriser le DNS sortant que depuis le serveur DNS interne et journaliser les requêtes ;
   - mettre en place une détection du beaconing et des sous-domaines anormaux (IDS, SIEM, DNS protégé).

Grille d'évaluation suggérée (sur 30)

Élément	Points	Critère de réussite
Partie A complétée	4	Captures montrées ; différence HTTP et HTTPS expliquée
Questions 1 à 5	4	Réponses exactes et commandes fournies
Questions 6 et 7	6	Ports ouverts justifiés par le filtre SYN-ACK ; rôle du RST expliqué
Questions 8 et 9	5	Mot de passe décodé ; calcul du port 50001 montré
Questions 10 et 11	6	Drapeau complet décodé ; beaconing nommé et mesuré
Question 12	1	Fichier remis, 80 paquets
Rapport	4	Chronologie, IoC, trois recommandations priorisées

Suggestion : retirer un demi-point par question sans la commande utilisée. Dans l'esprit du cours, la démarche compte autant que la réponse.

Annexe — Faire varier le lab

Le script `generer_capture.py` produit une capture différente si l'on modifie, avant de le relancer :

- `random.seed(420)` : change les numéros de séquence, les ports clients et l'ordre du balayage ;
- `secret` : le drapeau exfiltré par DNS ;
- `ouverts` : les ports ouverts sur SRV-FICHIERS ;
- `body` et les lignes `USER` / `PASS` : les identifiants exposés.

```
pip install scapy
python3 generer_capture.py
```

Après toute modification, il faut refaire les vérifications de ce corrigé : les heures, les comptes de paquets et la chaîne base32 changent.